for循环和fork...join_none

循环变量的调整

- 在for循环中，循环变量的调整值是当前次循环结束时自动按照**for**循环中描述的调整方式进行变更，并不是在**for**结构开始就进行变化的。

```
`timescale 1 ns / 1 ps
module top_tb;
initial begin
    fork
        $display("in fork 1st");
        $display("in fork 2nd");
        $display("in fork 3rd");
    join_none
    $display("After fork...join_none");
end
endmodule
```

硅芯思见

仿真执行过程	仿真结果
fork创建3个显示语句的线程	-
触发fork...join_none后的线程	“After fork...join_none”
执行fork创建的3个显示语句的线程	“in fork 1st”

硅芯思见

- 首先针对其中的语句创建三个线程，但是创建后并不会马上执行，而是触发了fork...join_none之后语句后才会执行已经创建的三个线程，所以从仿真结果中我们可以看到，首先显示了fork...join_none之后的显示语句，然后才显示了fork...join_none之中的三条显示语句。**可见fork...join_none中创建的线程并不是立刻执行，而是只有触发了其后的线程之后才会被执行。**

```
`timescale 1 ns / 1 ps
module top_tb;
initial begin
    for (int i = 0;i < 3;i++)
    fork
        $display("in fork : i is %0d",i);
    join_none
    $display("After fork...join_none");
end
endmodule
```

硅芯思见

循环变量值	仿真执行	仿真结果
0	for(...)语句	-
0	创建第一个显示语句的线程	-
0	循环变量i++	-
1	创建第二个显示语句的线程	-
1	循环变量i++	-
2	创建第三个显示语句的线程	-
2	循环变量i++	-
3	循环条件不满足，循环体不执行	-
3	执行$display(“After fork...join_none”);	After fork...join_none
3	执行创建的第一个线程	in fork : i is 3
3	执行创建的第二个线程	in fork : i is 3
3	执行创建的第三个线程	in fork : i is 3

硅芯思见

- for循环每执行一次就会通过fork...join_none创建一个线程，但是这些创建的线程并不会创建了就立刻执行，而是在fork...join_none之后语句被触发后才一并执行。因为这些线程使用的都是同一个变量i，即for循环的循环变量，所以当3次循环结束时，还未执行创建的进程时，循环变量已经变成了3

```
`timescale 1 ns / 1 ps
module top_tb;
initial begin
    for (int i = 0;i < 3;i++)
    fork
        automatic int j = i;
        $display("in fork : j is %0d",j);
    join_none
    $display("After fork...join_none");
end
endmodule
```

硅芯思见

循环变量值	线程1 j值	线程2 j值	线程3 j值	仿真执行	仿真结果
0	0	0	0	for(...)语句	-
0	0	0	0	创建第一个显示语句的线程	-
0	0	0	0	循环变量i++	-
1	0	1	0	创建第二个显示语句的线程	-
1	0	1	0	循环变量i++	-
2	0	1	2	创建第三个显示语句的线程	-
2	0	1	2	循环变量i++	-
3	0	1	2	循环条件不满足，循环体不执行	-
3	0	1	2	执行Sdisplay(“After fork...join_none”);	After fork...join_none
3	0	1	2	执行创建的第一个线程	in fork : j is 0
3	0	1	2	执行创建的第二个线程	in fork : j is 1
3	0	1	2	执行创建的第三个线程	in fork : j is 2

硅芯思见

- 在fork...join_none中定义了一个动态变量j，这样每执行一次for循环，fork...join_none创建一次对应的线程，并且因为每次创建的线程中j都是动态的，即每个线程中的j都属于各线程本身，且互相不影响，所以在每次创建对应线程时，当前次创建的线程中的动态变量j都会保存当前次循环变量i的值。

- *动态变量在声明的同时进行了赋值操作，因为声明性语句在其所在的过程块中将会最先执行，所以此时对于该变量的赋值操作在进行声明时也一并完成。*

```
`timescale 1 ns / 1 ps
module top_tb;
initial begin
    for (int i = 0;i < 3;i++)
    fork
        automatic int j;
        j = i;
        $display("in fork : j is %0d",j);
    join_none
    $display("After fork...join_none");
end
endmodule
```

硅芯思见

循环变量值	线程1 j值	线程2 j值	线程3 j值	仿真执行	仿真结果
0	0	0	0	for(...)语句	-
0	0	0	0	创建第一个显示语句的线程	-
0	0	0	0	循环变量i++	-
1	0	0	0	创建第二个显示语句的线程	-
1	0	0	0	循环变量i++	-
2	0	0	0	创建第三个显示语句的线程	-
2	0	0	0	循环变量i++	-
3	0	0	0	循环条件不满足，循环体不执行	-
3	0	0	0	执行$display(“After fork...join_none”);	After fork...join_none
3	3	0	0	执行创建的第一个线程	in fork : j is 3
3	3	3	0	执行创建的第二个线程	in fork : j is 3
3	3	3	3	执行创建的第三个线程	in fork : j is 3

硅芯思见

- j声明的时候并没有给j赋初值，所以在每次创建线程时j并没有被赋值。而j被赋值为i的操作实际上算为fork...join_none创建的一个线程，所以**j获得的i实际上是fork...join_none已经完成所有线程的创建了**。在fork...join_none后语句执行后，依次启动已经创建的线程，在这些线程执行的时候，循环变量i的值已经是3了，所以每个启动的线程在执行时，进行对j赋值时，j获得的是循环变量最后的值，即所有3个线程获得的i都是一个值。

```
1 `timescale 1 ns / 1 ps
2 module top_tb;
3 initial begin
4     for (int i = 0;i < 3;i++)
5     fork
6         int j = i;
7         $display("in fork : j is %0d",j);
8     join_none
9     $display("After fork...join_none");
10 end
11 endmodule
```

硅芯思见

```
Top Level Modules:
       top_tb
TimeScale is 1 ns / 1 ps

Error-[SV-IRTAV] Illegal reference to automatic variable
top_tb.sv, 6
"top_tb.0.i"
  Hierarchical reference to automatic variable 'i' is not legal.
  Declared at:
  "top_tb.sv", 4

1 error
CPU time: .240 seconds to compile
```

硅芯思见

- **静态变量在声明的同时进行了初始化，此操作应该在仿真开始就执行(静态变量的初始化在仿真开始时进行)，而仿真开始时，实际上for循环并没有执行，即此时i还不存在，所以此时编译错误.**
- 如果将声明和初始化分开，这时仿真结果与将automatic变量j的声明和初始化分开效果一样，即每个线程获得的i的值相同，都是i最后一次循环执行完后的递增值3.

```
`timescale 1 ns / 1 ps
module top_tb;
initial begin
    for (int i = 0;i < 3;)
    fork
        i++;
        $display("in fork : i is %0d",i);
    join_none
    $display("After fork...join_none");
end
endmodule
```

硅芯思见

DEAD LOOP！

并且在仿真的时候会提示如下：

```
Parsing design file 'top_tb.sv'

Warning-[FLWSA] For loop without step assignment
top_tb.sv, 4
  The SystemVerilog LRM requires that at least one 'for_step_assignment' to be
  present in 'for_step' of a 'for' loop.

CPU time: .171 seconds to compile
```

硅芯思见

- 仿真进入死循环，为什么会这样呢？这是因为for循环体中的fork...join_none中创建的线程会在fork...join_none后的语句执行后才会启动，而此时循环本身并没有实现循环变量的自动调整(其自动调整的操作已经被包含在了要启动的线程中，所以此时for循环的循环变量将一直保持0，即一直满足循环执行的条件，进入了死循环)，导致了for循环后的语句也不会执行。

注意事项

- for循环进行循环时，其循环变量是在当前次循环结束后才会按照调整表达式变化，当然for循环变量的调整表达式可以出现在循环体内部从而实现循环变量的手动调整；
- **fork...join_none开启的线程在创建后并不会立刻执行，而是在触发了fork...join_none后的语句之后才会一并启动**；
- 变量的声明会在其所在语句块中最先执行；
- 动态变量仅存在于其所在线程中，多个线程中的动态变量互相不产生影响；